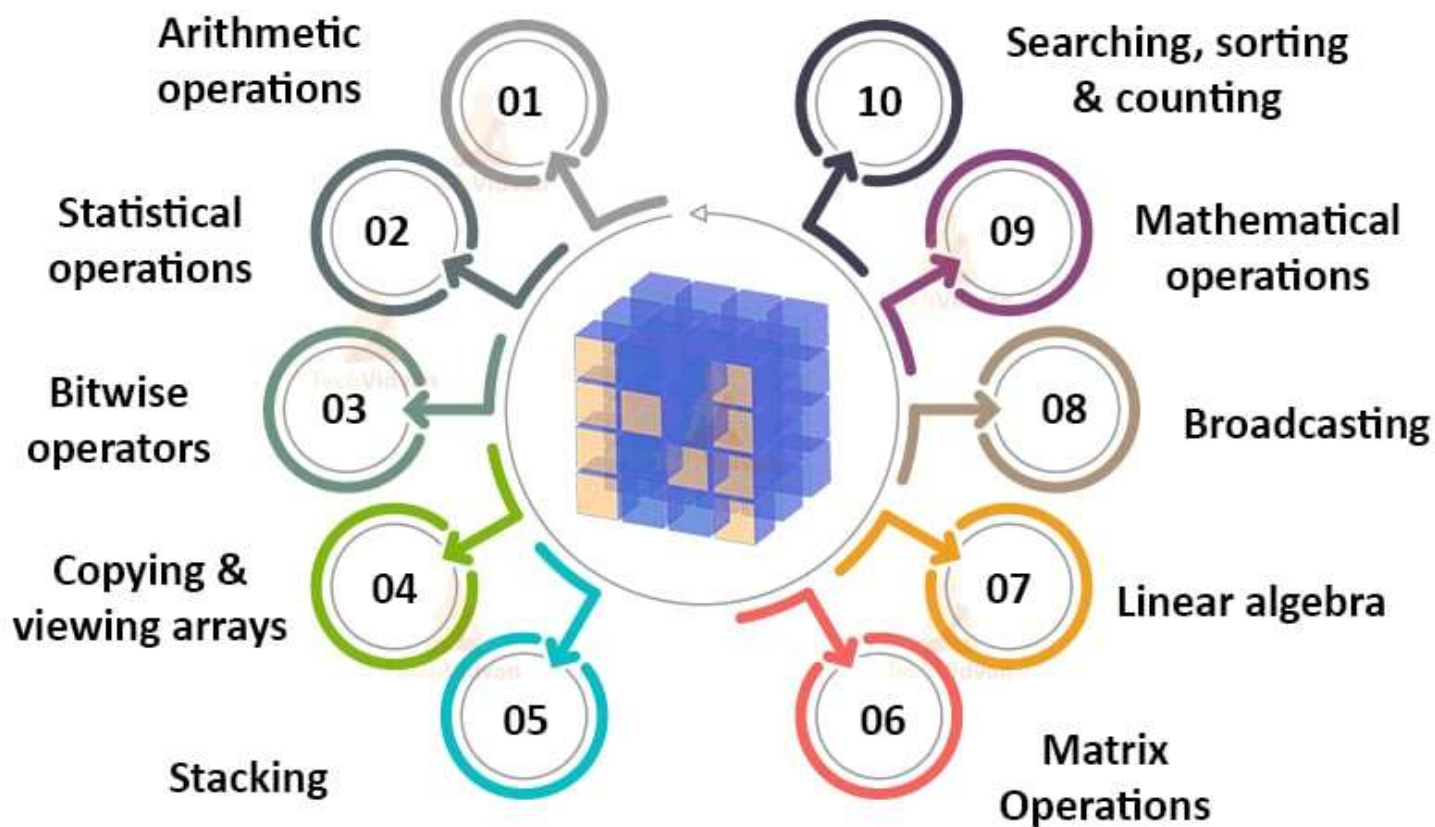


Numpy

NumPy, short for Numerical Python, is a fundamental open-source library in Python for scientific computing. It provides support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate efficiently on these arrays.

Uses of NumPy



1. Importing NumPy

```
import numpy as np
```

2. Creating Arrays

```
a = np.array([1, 2, 3])          # 1D array
```

```
b = np.array([[1, 2], [3, 4]])  # 2D array
```

```
zeros = np.zeros((2, 3))        # array of zeros
```

```
ones = np.ones((2, 3))          # array of ones
```

```
full = np.full((2, 2), 7)       # array filled with 7
```

```
eye = np.eye(3)                 # identity matrix
```

```
arange = np.arange(0, 10, 2)    # range with step
```

```
linspace = np.linspace(0, 1, 5) # equally spaced numbers
```

Array Attributes

#3. Array Attributes

```
arr = np.array([[1,2,3],[4,5,6]])
```

```
print(arr.shape) # (2,3)
```

```
print(arr.ndim) # 2
```

```
print(arr.size) # 6
```

```
print(arr.dtype) # int64 (depends on system)
```

```
print(arr.itemsize) # bytes per element
```

Indexing & Slicing

4. Indexing & Slicing

```
arr = np.array([10,20,30,40,50])
```

```
print(arr[0])    # 10
```

```
print(arr[-1])   # 50
```

```
print(arr[1:4])   # [20 30 40]
```

```
mat = np.array([[1,2,3],[4,5,6]])
```

```
print(mat[0,1])   # 2
```

```
print(mat[:,1])   # column [2 5]
```

```
print(mat[1,:])   # row [4 5 6]
```

Mathematical Operations

#5. Mathematical Operations

```
x = np.array([1,2,3])
```

```
y = np.array([4,5,6])
```

```
print(x + y)    # [5 7 9]
```

```
print(x - y)    # [-3 -3 -3]
```

```
print(x * y)    # [ 4 10 18]
```

```
print(x / y)    # [0.25 0.4 0.5 ]
```

```
print(np.power(x, 2)) # [1 4 9]
```

Aggregate Functions

```
arr = np.array([1,2,3,4,5])
```

```
print(arr.sum())    # 15
```

```
print(arr.mean())   # 3.0
```

```
print(arr.min())    # 1
```

```
print(arr.max())    # 5
```

```
print(arr.std())    # 1.414..
```

```
print(arr.var())    # 2.0
```


Reshaping

```
arr = np.arange(1,7)    # [1 2 3 4 5 6]
```

```
reshaped = arr.reshape(2,3)
```

```
# [[1 2 3]
```

```
#  [4 5 6]]
```

```
flat = reshaped.ravel() # back to 1D
```

Stacking & Splitting

NumPy provides various functions for both stacking (joining) and splitting arrays, allowing for flexible manipulation of data structures.

```
a = np.array([[1,2],[3,4]])
```

```
b = np.array([[5,6],[7,8]])
```

```
print(np.hstack((a,b)))
```

```
# [[1 2 5 6]
```

```
# [3 4 7 8]]
```

Stacking & Splitting

```
print(np.vstack((a,b)))
```

```
# [[1 2]
```

```
# [3 4]
```

```
# [5 6]
```

```
# [7 8]]
```

```
arr = np.array([1,2,3,4,5,6])
```

```
print(np.split(arr, 3)) # [array([1,2]), array([3,4]), array([5,6])]
```

Random Module

```
print(np.random.rand(3))      # random floats [0,1)
print(np.random.randn(2,2))   # normal distribution
print(np.random.randint(1,10,5)) # random ints
print(np.random.choice([1,2,3])) # random element
```

Linear Algebra

```
A = np.array([[1,2],[3,4]])
```

```
B = np.array([[5,6],[7,8]])
```

```
print(np.dot(A,B)) # matrix multiplication
```

```
print(np.linalg.inv(A)) # inverse
```

```
print(np.linalg.det(A)) # determinant
```

```
print(np.linalg.eig(A)) # eigenvalues & vectors
```

Useful Functions

```
arr = np.array([1,2,3,4,5])
```

```
print(np.unique(arr))      # unique values
```

```
print(np.sort(arr))       # sort
```

```
print(np.where(arr>2))     # indices where condition holds
```

```
print(np.clip(arr, 2, 4))  # limit values [2 2 3 4 4]
```

```
print(np.argmax(arr))     # index of max → 4
```

```
print(np.argmin(arr))     # index of min → 0
```

Pandas

Pandas is a popular open-source software library written for the Python programming language, specifically designed for data manipulation and analysis. It provides powerful and flexible data structures, primarily the DataFrame and Series, which are optimized for working with "relational" or "labeled" data.

Introduction

```
import numpy as np
```

```
pandas = data analysis library
```

```
#Works with Series (1D) and DataFrame (2D)
```


Pandas Data Structures

```
#Series (1D)
```

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
```

```
print(s)
```

```
# DataFrame (2D)
```

```
data = {"Name": ["Alice", "Bob", "Charlie"], "Age": [25, 30, 35]}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Reading & Writing Data

```
df = pd.read_csv("data.csv")    # Read CSV
```

```
df.to_csv("output.csv", index=False) # Save CSV
```

```
df = pd.read_excel("data.xlsx")  # Read Excel
```

```
df.to_excel("output.xlsx", index=False)
```

Basic Info & Exploration

`print(df.head())` # first 5 rows

`print(df.tail(3))` # last 3 rows

`print(df.shape)` # rows, cols

`print(df.info())` # data types

`print(df.describe())` # summary stats

`print(df.columns)` # column names

`print(df.dtypes)` # column types

Selecting Data

✓ By column

```
print(df["Name"])
```

```
print(df[["Name", "Age"]])
```

✓ By row

```
print(df.loc[0])    # by index label
```

```
print(df.iloc[0])   # by position
```

✓ Conditional selection

```
print(df[df["Age"] > 30])
```

Adding / Modifying Columns

```
df["Salary"] = [5000, 6000, 7000]    # add new
```

```
df["Age+5"] = df["Age"] + 5           # derived
```

Deleting Rows/Columns

```
df.drop("Salary", axis=1, inplace=True) # drop column
```

```
df.drop(0, axis=0, inplace=True) # drop row
```

Handling Missing Data

`df.dropna()` # drop missing

`df.fillna(0)` # replace with 0

`df["Age"].fillna(df["Age"].mean(), inplace=True)` # replace with mean

Sorting

```
df.sort_values("Age")          # ascending
```

```
df.sort_values("Age", ascending=False)  # descending
```


Grouping & Aggregation

```
df.groupby("Name")["Age"].mean()
```

```
df.groupby("Name").agg({"Age": "max"})
```

Merging & Joining (SQL style)

```
pd.merge(df1, df2, on="id", how="inner") # inner join
```

```
df1.join(df2, how="outer") # index-based join
```

Concatenation & Append

`pd.concat([df1, df2])` # stack vertically

`df1.append(df2)` # (deprecated, use concat)

Pivot & Crosstab

#Reshapes data: turns **unique values of a column** into **columns**.

```
pd.pivot_table(df, values="Age", index="Name", aggfunc="mean")
```

#Used to compute **frequency tables** (counts).

```
pd.crosstab(df["Name"], df["Age"])
```

Apply & Lambda

```
df["Age2"] = df["Age"].apply(lambda x: x*2)
```

String Operations

```
df["Name"].str.upper()
```

```
df["Name"].str.contains("A")
```

Date & Time

```
df["date"] = pd.to_datetime(["2025-01-01", "2025-02-01", "2025-03-01"])
```

```
df["year"] = df["date"].dt.year
```

Window Functions

```
df["rolling_mean"] = df["Age"].rolling(2).mean()
```

```
#Rolling Mean (Moving Average)
```

```
df["Rolling_Mean"] = df["Sales"].rolling(window=3).mean()
```

```
print(df)
```

```
#Like SQL RANK, DENSE_RANK, ROW_NUMBER.
```

```
df["Rank"] = df["Sales"].rank(method="dense", ascending=False)
```

```
print(df)
```


Exporting

```
df.to_csv("out.csv")
```

```
df.to_json("out.json")
```

```
df.to_excel("out.xlsx")
```

Visualization (with matplotlib)

```
import matplotlib.pyplot as plt  
df["Age"].plot(kind="hist")  
plt.show()
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Create a sample DataFrame
data = {
    "Product": ["Laptop", "Mobile", "Tablet", "Headphone", "Camera"],
    "Sales": [120, 300, 180, 90, 150]
}
df = pd.DataFrame(data)
print(df)

# Plot Bar Graph
plt.bar(df["Product"], df["Sales"], color="skyblue", edgecolor="black")

plt.title("Product Sales")
plt.xlabel("Product")
plt.ylabel("Sales Units")
plt.show()
```